Lesson Guru Odisha — production, Play Store & updates guide

Written 26 Oct 2026; updated for app v1.3.0. It covers four things:

1. What the backend and database should be for long-term production
2. Whether you can publish on Google Play, and the step-by-step process
3. How to update the website, backend and app in future
4. What the admin panel does, with screenshots

---

0. Short answers

- **Backend and database:** keep what you have, Next.js with MongoDB. Don't rewrite it. For a lifelong product, move it onto infrastructure you control:
  - **MongoDB Atlas** in Mumbai, with point-in-time backups;
  - the app in a **Docker container** on Google Cloud Run (or a similar host) in Mumbai;
  - **Cloudflare R2** for files, which you already use;
  - separate staging and production;
  - monitoring and automatic tests on every deploy.
- **Play Store:** yes. The payments question (§2.1) is settled: **v1.3.0 is a consumption-only app (option A)**. Nothing is sold inside the app; teachers buy on the website and the app unlocks what their account has. Target API 36, account deletion and the privacy policy are also done.
- **Existing teachers (17,000+):** they log in to the app with the same mobile number and password, or WhatsApp OTP. It is the same website, backend and database, so there is no new registration and subscriptions carry over. Note: the site allows one active login per account, so logging in on the app signs that teacher out of the browser (and the other way round).
- **Updates:** most changes are website/backend deploys, and they reach the app immediately. Only changes to the app shell (its own screens, native code, version) need a new Play upload, which takes about 15 minutes.

---

1. Backend & database for long-term production

1.1 What runs today

Piece	Today	Notes
Web + API	One Next.js 14 app (`app/page.js` 25k lines, `app/api/[[...path]]/route.js` 26k lines)	Works. The size makes changes risky without tests.
Database	MongoDB, managed by Emergent (separate preview and live DBs)	Several RCAs came from preview/live DB mix-ups and failed builds
Files	Cloudflare R2 (via a Worker gateway) + Cloudinary (legacy)	R2 migration half-done, see `R2_LIVE_SWITCH_RUNBOOK.md`
Scheduled jobs	Emergent crons → HTTP endpoints (`.emergent/crons.yml`)	Easy to move: they're plain HTTP calls
Payments	Razorpay	
WhatsApp	BulkyMarketing (templates + OTP)	UltraMsg retired
AI	Gemini / OpenAI	
PDFs	Server-side Chromium (`@sparticuz/chromium`, puppeteer)	Needs a container host; serverless size limits break it

1.2 Recommendation

Keep Next.js + MongoDB. Your data is document-shaped (lessons, chapters, plans, orders), the code works, and a rewrite would cost months and add risk. What changes is *where* it runs and *how* you operate it.

Layer	Recommended	Why
Database	**MongoDB Atlas, Mumbai region (AWS ap-south-1 or GCP asia-south1), dedicated M10 → M20 as you grow**	Point-in-time restore (go back to any minute), automatic backups, alerts, separate projects for staging and production
App hosting	**Google Cloud Run (asia-south1)** running a Docker image of the Next.js app, min 1 instance. Alternatives: Render or Railway (simpler), or one AWS Lightsail/DigitalOcean VPS in Mumbai with Docker (cheapest, more hands-on)	Containers keep Chromium PDF generation working. Scales with traffic and you control the deploys. Avoid pure serverless (e.g. Vercel functions): body-size limits break 25 MB uploads, and Chromium is awkward there
Files	**Cloudflare R2** (finish the runbook; once off Emergent use the direct S3 mode and drop the gateway)	No egress fees, already in place
CDN / DNS / protection	**Cloudflare** in front of lessonguruodisha.in	Caching, DDoS protection, free TLS
Scheduled jobs	**Cloud Scheduler** (or the host's cron) hitting the same URLs in `.emergent/crons.yml`	No code changes
Monitoring	Uptime check on `/api/health` (UptimeRobot / Better Stack), **Sentry** for errors, Atlas alerts, daily look at **Admin → 📲 OTP Health**	Find problems before teachers do
Source & CI	GitHub as the source of truth. GitHub Actions on every push runs `yarn install --frozen-lockfile`, `next build`, the backend tests (`tests/cloud`) and the app launch test (`android-app/launch-test`), then deploys staging; production deploys only after manual approval	Stops the lockfile and prerender failures in your RCAs from reaching production

Rough monthly cost (verify current prices):

Item	Estimate
MongoDB Atlas M10, Mumbai	≈ US$60–70
Cloud Run, 1 always-on instance + traffic	≈ US$20–60
Cloudflare R2 (100 GB) + Workers	≈ US$2–5
Sentry + uptime monitor	free tiers to start
Total	**≈ ₹8,000–12,000 / month**, growing with traffic

1.3 Staying on Emergent vs moving

- **Staying on Emergent** is fine short-term: it works, and someone else runs it. The risks for a "lifelong" product are limited control over backups and restore, no CLI on live, and being tied to one platform. If you stay, at least: export a backup of the live DB every week to R2, and use the admin "Snapshot" before every risky change (as the runbook says).
- **Moving** (recommended within the next few months):
  1. Create Atlas projects `lgo-staging` and `lgo-prod` (Mumbai). Use `mongodump` from Emergent live, then `mongorestore` into `lgo-prod`; rehearse on staging first.
  2. Add a `Dockerfile` (Node 20 + Chromium deps) and deploy the image to Cloud Run for staging. Set every env var from Emergent there; get the list from Emergent's settings.
  3. Test staging end to end: login with OTP, a Razorpay test payment, PDF downloads, Cloud Library upload.
  4. Pick a quiet night, put the site in read-only mode for ~30 minutes, do a final dump/restore, switch Cloudflare DNS to Cloud Run, and watch `/api/health` and the OTP Health screen.
  5. Keep Emergent untouched for 2 weeks as a fallback, then retire it.

1.4 Rules for the database (whatever the host)

- Backups: point-in-time restore on, plus a weekly `mongodump` copied to R2 and kept 90 days.
- Never point preview/staging code at the production DB (this caused past incidents). Use separate Atlas projects with different passwords.
- Add indexes when you add queries. The new collections create their own: `cloudFiles`, `cloudFileChunks`, `accountDeletionRequests`.

- Secrets live only in the host's secret manager. Set `CLOUD_LINK_SECRET` (new), and never commit `.env`.

---

2. Google Play — can you publish? Yes. Here's how.

2.1 ✅ Decided: how Premium is paid for (option A, done in v1.3.0)

Google Play's **Payments policy** says that apps selling digital content or subscriptions (Premium plans, store packs, AI credits) must use **Google Play Billing**. India has an exception: Google's alternative-billing program, under which you may offer your own payment method **alongside** Google Play Billing. Today the app opens Razorpay checkout inside the app. Submitted as-is, it can be rejected or later removed. Choose one:

Option	What you do	Google's fee	Effort
A. Consumption-only app (recommended to launch)	Inside the app, hide all buying: Premium prices, "Go Premium", plans, store checkout. Teachers buy on the website as today; the app unlocks content for paying accounts	0%	Small. I can add an "app mode" that hides purchase UI only inside the app (it can recognise the app by its `LessonGuruApp/` user agent)
B. India alternative billing	Offer Google Play Billing **and** Razorpay side by side in the app (user's choice)	≈11% on purchases through Razorpay, ≈15% through Google Play (Google's India program: 4% lower when users choose your payment method)	Large: Play Billing Library, purchase verification on your server, enrolment and reporting to Google
C. Google Play Billing only	Replace Razorpay inside the app with Google Play Billing	15% on subscriptions	Large

Recommendation: **launch with A**, which is quick, policy-safe and keeps 100% of revenue. Revisit B once the app has users and you want in-app purchases. Google's rules on mentioning external purchase options are strict and change by country. Read the current pages before deciding: Payments policy and India billing changes.

2.2 Already handled in v1.3.0

- ✅ **Target API 36 (Android 16)**: required for new apps and updates since 31 Aug 2026 (Google). Includes the Android 16 back-gesture change.
- ✅ **Account deletion in the app and on the web**: Profile → Delete account, and `https://lessonguruodisha.in/delete-account`. Admins process requests in Admin → 🗑️ Account Deletions within 30 days.
- ✅ **Privacy policy updated**: now covers the Cloud Library, the WhatsApp provider and account deletion.
- ✅ **Consumption-only app (option A)**. Inside the app (detected by the `LessonGuruApp/` user agent):
  - Premium plans become a status-only **My Plan** page (`/my-plan`): "Premium active · valid until ...", or "Free plan" with "Purchases are not available in the app";
  - hidden: prices, "View Plans", Upgrade/Renew buttons, renewal reminders and pop-ups, home promo cards and admin promo ads, store/assessment-store cards, "Buy More Credits", paid answer-key unlocks and bundles, Ajira add-on purchase, and the buy-credits pop-up;
  - the app itself refuses to open `/plans` , `/store` (except **My Library** and **orders**), `/assessment` and `/my-qr-payments` , and shows "Not available in the app";
  - Razorpay checkout cannot open inside the app, even from a button that was missed.

  On the website nothing changes: teachers buy exactly as today, and the purchase shows up in the app after they reopen it. Don't tell teachers *inside the app* to go and pay on the website. Google restricts that too. WhatsApp messages and the website are fine.
- ✅ **Signed AAB** (`android-app/dist/LessonGuruOdisha-v1.3.0.aab`), listing text, icon, feature graphic and 8 phone screenshots (`android-app/dist/store/`).

2.3 Step-by-step release

Step 1: Developer account (once)

1. Go to https://play.google.com/console and sign up with the business Google account you'll keep forever (not a personal Gmail that might be lost).
2. Choose **Organization** (recommended for a business). You need a **D-U-N-S number** for "Teacher Utility Tool Kit", free from Dun & Bradstreet; it takes a few days. Pay the one-time **US$25** fee and complete identity verification.

    - With a *personal* account created after Nov 2023, Google requires a **closed test with at least 12 testers for 14 days** before you can publish to production. Organization accounts skip this.
3. Verify the developer email, phone and website (lessonguruodisha.in).

Step 2: Create the app

1. Play Console → **Create app**: name *Lesson Guru Odisha*, default language *English (India) – en-IN*, *App*, *Free*. Accept the declarations.
2. **App signing:** keep **Play App Signing** on (the default). Your file `android-app/keystore/lessonguru-upload.p12` becomes the **upload key**. Back up that file and `keystore.properties` in two safe places. If you lose them, you'll need Google support to reset the key.

Step 3: App content (Policy → App content); answer every item

Item	What to enter
Privacy policy	`https://lessonguruodisha.in/privacy`
App access	**"All or some functionality is restricted"**. Give reviewers a test account with a **mobile + password** login (they can't receive WhatsApp OTPs) and Premium active. Create it in Admin → Users
Ads	**Yes** if you show promotions from Admin → 📢 Advertisements (Google counts house ads); otherwise No
Content rating	Fill in the IARC questionnaire: category *Reference, News, or Educational*; no violence, sexual content, gambling or user-to-user chat. Usually rated *Everyone / 3+*
Target audience	**18 and over** (the app is for teachers). This keeps you out of the stricter Families programme
News app	No
Data safety	See the table below
Government apps	Declare that you are **not** a government entity. The listing description already says the app is not affiliated with the Govt of Odisha/OSEPA/SCERT; keep that line
Financial features	None
Account deletion	`https://lessonguruodisha.in/delete-account`

Data safety answers (based on what the code actually does; review with your own knowledge):

Data type	Collected?	Purpose	Notes
Name	Yes	Account management, app functionality	Required
Phone number	Yes	Account management, app functionality, communications (WhatsApp OTP)	Required
Email	Yes, if the teacher provides it	Account management	Optional
Purchase history	Yes	App functionality, account management	Subscription/orders
Payment info	Entered on Razorpay's page (processor)	Payments	You never store card/UPI details
Files & docs	Yes (Cloud Library)	App functionality	Optional; teacher can delete
App interactions	Yes (lessons viewed, history, favourites)	App functionality, analytics	
Crash logs	**No** (the in-app report is sent only if the teacher chooses to share it on WhatsApp)		
Location, contacts, photos, device IDs	No		

Also answer: data is **encrypted in transit** (HTTPS): Yes. Users **can request deletion**: Yes, with the URL above. Data shared with third parties: **No** (Razorpay, WhatsApp and cloud hosts are *service providers*, which Google doesn't count as sharing).

Step 4: Store listing Main store listing: copy `android-app/dist/store/listing.md`, upload the icon, feature graphic and 8 screenshots from `android-app/dist/store/`. Category **Education**.

Step 5: Test, then release

1. **Testing → Internal testing → Create release** → upload `LessonGuruOdisha-v1.3.0.aab` → add your own Gmail as a tester → install from the Play link on a real phone. Check:
   - login (OTP and password);
   - My Plan shows your plan, and no prices or buy buttons appear anywhere;
   - a Premium account opens Premium lessons;
   - PDF download → "Saved to phone & Cloud Library";
   - Back button and gesture;
   - Delete account page opens.
2. (Personal accounts only) **Closed testing** with 12+ testers for 14 days.
3. **Production → Create release** → same AAB → release notes → **staged rollout 20%**, watch Play Console → Android vitals (crashes) for 2 days, then 100%.
4. The first review usually takes a few days. Answer any policy email promptly.

Step 6: After the first release

- Play Console → **Setup → App integrity → App signing** → copy the **SHA-256 of the app signing key** and add it to `public/.well-known/assetlinks.json` next to the upload key that's already there. Deploy the website, and `lessonguruodisha.in` links will open straight in the app.

---

3. How to update in future

3.1 What needs which kind of update

You change…	What to do	Teachers get it
Website pages, API, prices, lesson content, admin features	Deploy the website as usual	Immediately, on web **and** in the app (the app loads the live site)
App hub screens (`android-app/src/main/assets/app`), native code, app icon, permissions	New app version via Play	After the Play update installs

3.2 Releasing a new app version (≈15 minutes)

1. Make the change in `android-app/`.
2. In `android-app/build-tools/build.py`, raise `VERSION_CODE` by 1 (it must always go up) and set `VERSION_NAME` (e.g. `1.3.0`).
3. Build: `python3 build-tools/build.py --tools <tools folder>`. This produces the signed APK and AAB in `dist/` (see `android-app/README.md`).
4. Test:
   - `cd tests/cloud && TEST_DEPS=../.. node --import ./register.mjs --test *.test.mjs` (backend: 23 tests);
   - `cd android-app/launch-test && gradle test -Ptools=<tools folder>` (app on Android 7–16: 50 tests);
   - install the APK on one real phone.
5. Play Console → Production → Create release → upload the new `.aab` → staged rollout.

3.3 Yearly / recurring

- **Every August Google raises the required target API.** Next: API 37 by about 31 Aug 2027. Change `TARGET_SDK` in `res_build.py`, read Android's behaviour changes for that version, run the launch tests (add the new version to `@Config(sdk=…)`), and release.
- Renew the domain, Razorpay KYC, the BulkyMarketing wallet and WhatsApp templates on time.
- Keep the privacy policy and Data safety form in line with any new data you collect.
- Keep the upload keystore safe. You need it for every update.

---

4. Admin panel (what each section does)

Screenshots: `android-app/dist/admin-screenshots/` (58 sections, sample data) and the overview sheet `android-app/dist/LessonGuruOdisha-admin-overview.png`. Admin works on phone or desktop at `lessonguruodisha.in/admin`.

Money & users

- 📊 Analytics: users, active subscriptions, lessons, revenue, DAU, popular lessons and books
- 💰 Revenue: daily/monthly revenue, plan breakdown, renewal mode (extend/replace), activation log
- 📱 QR Payments: verify manual UPI-QR payments (UTR), approve or reject

- 👥 Users / 🗓️ User Profiles: search, edit subscriptions, reset passwords, suspend, registration stats
- 💎 Plans / 🔥 Offers / 🔄 Subscription Recovery / 🔁 Renewal Funnel: pricing, offers, win-back
- 🎁 Referrals / ⭐ Reviews: leaderboard, reward reversal, review moderation
- 💎 AI Credits: AI wallet sales, Gemini usage and cost, margins

Messaging

- 📱 WhatsApp Campaigns / 📣 Message Queue / 📣 Prompt / 📩 Inbox: campaigns, drip, support messages
- 📲 **OTP Health (new)**: is WhatsApp OTP working right now? Shows key and template status, the wallet estimate, 24-hour sent/failed/blocked counts, the provider's exact reply for each failure, and a **Send test OTP** button

Content

- Lessons: ➕ Add Lesson, 📄 HTML Import, ⚡ CSV/JSON, 📑 Bulk Chapters, 🔢 Bulk Periods, 🗂️ Curriculum (+ Backfill), 📅 Academic Pacing
- Panchapadi: 📘 Panchapadi (ABP), 📘 Panchapadi HTML, 🌅 Ajira Pathya Jojana, ⭐ Short Lesson Plans
- Assessment: 🎯 Assessments, 📋 Instant Papers, 📝 Answer Keys, 🛒 Assessment Store, 📚 QB Import, 📝 Q&A Worksheets, 🧠 MCM Test, 📘 MCM Notes
- LO codes: 📗 LO Master, 🎯 LO Code Correction, 🔧 LO Reconcile, 🈳 WS Language
- Books & files: 📕 Books & PDFs, ☁️ Drive Books, 📚 Sections Upload, 📚 Resources, 🗄️ R2 Migration
- AI: 🤖 AI Lesson Generator, ✨ AI Studio, 🎓 Class 9–10 5E
- Store & other: 🛍️ Store, 📋 Teacher Observation, 🚀 Landing Page, 📢 Advertisements, 🔐 Security, 📄 Doc Security, 📈 WS Analytics, 📋 Logs

Privacy

- 🗑️ **Account Deletions (new)**: requests from the app/website. *Delete account* removes the teacher's Cloud Library files and anonymises the account (payment records kept for tax law). Process every request within 30 days; overdue ones are marked red.

Suggested routine

- Daily: QR Payments, OTP Health, Inbox
- Weekly: Revenue, Renewal Funnel, Account Deletions, Reviews
- Monthly: back up the DB, check the WhatsApp wallet and templates, review AI Credits margins

5. Fixes in this round, and how to confirm them on the live site

UPI not showing in the app. Cause: Razorpay hides UPI inside Android WebViews unless checkout is opened with `webview_intent: true`. v1.2.0 fixed that, but from v1.3.0 the app sells nothing (option A), so payments only happen on the website, where UPI works normally. If you later move to option B, open Razorpay checkout with `webview_intent: true` inside the app.

Deploy the website together with app v1.3.0. The app blocks store/plans pages and Razorpay on its own, but hiding prices and buy buttons on other pages (home cards, dashboard, credits, answer keys, Ajira) is done by the website code. Deploy the website first, then submit the app.

WhatsApp OTP not arriving. I couldn't reach your live server or BulkyMarketing from here, so this isn't diagnosed against live data yet. What changed:

- The OTP sender now treats "HTTP 200 but refused" replies (e.g. empty wallet) as failures.
- Every send records the provider's reply (OTP masked).
- The per-network limit was raised from 5 to 30 per hour; Jio/Airtel share IPs, so the old limit blocked unrelated teachers.
- New **Admin → 📲 OTP Health** screen.

After deploying the website, open OTP Health. It tells you which of these it is:

- keys missing;
- template not APPROVED;
- wallet empty;
- the provider rejecting sends (its reply is shown).

Use **Send test OTP** to your own number. If the provider accepts it but WhatsApp never delivers, the problem is on BulkyMarketing's side (template category, sending number quality, or wallet); raise it with them, quoting the `ref` shown.

Admin Inbox crash. `useEffect(load, …)` returned a Promise to React, which crashes when you leave the Inbox tab. Fixed in `app/page.js`.